▶ Notebook

Lecture 5 at a glance: four parts, seventeen sections

Part 1 · Where we start: the ELBO

1.1 What Lecture 1 gave us ▶
1.2 Today's question ▶

Part 2 · Autoencoders: squeeze, then rebuild

2.1 An image is 784 numbers ▶
2.2 Encoder, code, decoder ▶
2.3 The loss, and training ▶
2.4 What the code holds ▶

Part 3 · Can an autoencoder generate?

3.1 Decode random codes ▶
3.2 Where the codes are ▶
3.3 What we want instead ▶

Part 4 · The VAE: Lecture 1's ELBO, one image at a time

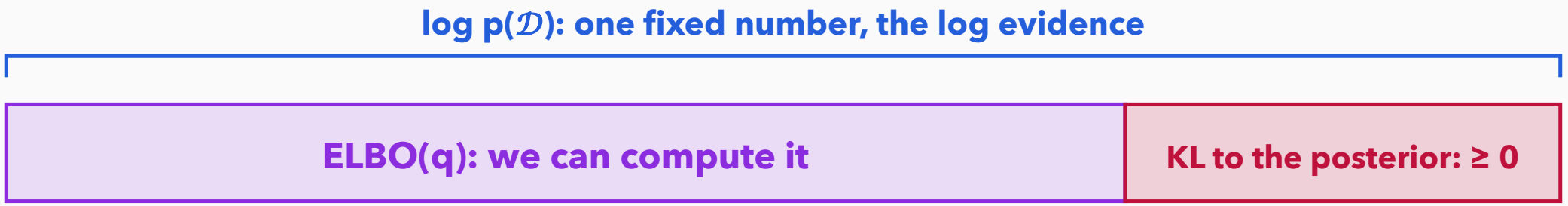
4.1 A model with a hidden code ▶
4.2 The same wall as Lecture 1 ▶
4.3 The ELBO for one image ▶
4.4 The loss, term by term ▶
4.5 Two terms pull against each other ▶
4.6 What the VAE learned ▶
4.7 The KL weight β ▶
4.8 Wrap-up ▶

> Four parts, seventeen numbered sections, about an hour. The ▶ Notebook link on every slide opens its section in Colab.

▶ Notebook

Part 1 · Where we start: the ELBO

1 · Where we start | 2 · Autoencoders | 3 · Can it generate? | 4 · The VAE



raise the ELBO, and the KL to the posterior must fall

1.1 What Lecture 1 gave us ▸

1.2 Today's question ▸

> Lecture 1 ended with one objective we can compute: the ELBO. Today it trains two networks that squeeze, rebuild and invent handwritten digits.

▶ Notebook 1.1

From Lecture 1: one objective we can compute

$$\log p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

log p($\mathcal{D}$) is one fixed number. Whatever the ELBO gains, the KL to the posterior loses.

$$\mathrm{ELBO}(q) = \underbrace{\mathbb{E}_{\theta\sim q}\big[\log p(\mathcal{D} \mid \theta)\big]}_{\text{fit the data}} - \underbrace{\mathrm{KL}\big(q(\theta) \,\|\, p(\theta)\big)}_{\text{stay near the prior}}$$

$$\text{train:} \quad \psi^* = \arg\max_{\psi} \ \mathrm{ELBO}(q_\psi) \qquad \text{loss} = -\mathrm{ELBO}$$

↗ Lecture 1, 4.1: the objective ↗ Lecture 1, 4.1: the ELBO in two terms

> The objective: tune q's knobs ψ to maximise the ELBO, by gradient descent on minus the ELBO. Today reuses this line unchanged.

▶ Notebook 1.1

Two tools from Lecture 1 we will use again

① A random draw that gradients pass through

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

The randomness sits in ε, so the gradient reaches μ and σ.

↗ Lecture 1, 3.5: reparameterisation

② The KL from a Gaussian to N(0, 1), by formula

$$\begin{aligned}\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, 1)\big) \\ = \tfrac{1}{2}\big(\mu^2 + \sigma^2 - \log \sigma^2 - 1\big)\end{aligned}$$

Zero only at μ = 0, σ = 1. With several code numbers, add one such term per number.

↗ Lecture 1, 2.6: diagonal Gaussians

> Tool two: the KL from a Gaussian to N(0, 1) has a formula, zero exactly when μ = 0 and σ = 1.

▶ Notebook 1.2

Real or generated? Six of these twelve digits are fake



Six are handwritten MNIST test digits. Six come from a network.

The fakes: numbers 2, 3, 5, 10, 11, 12, counting left to right, top to bottom.

By the end of the hour you will know how the network that made them was trained: a VAE with 16 numbers per code.

> The six outlined in red are generated. Some give themselves away with broken strokes; some fool most people.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 1.2

Today: two networks, then Lecture 1's objective

Part 2 · Autoencoder

One network squeezes a digit into 2 numbers; a second rebuilds it. The loss is squared error.

Part 3 · Can it generate?

Feed the second network random numbers instead. We will see what comes out, and why.

Part 4 · The VAE

A model with a hidden code, trained by maximising Lecture 1's ELBO, one image at a time.

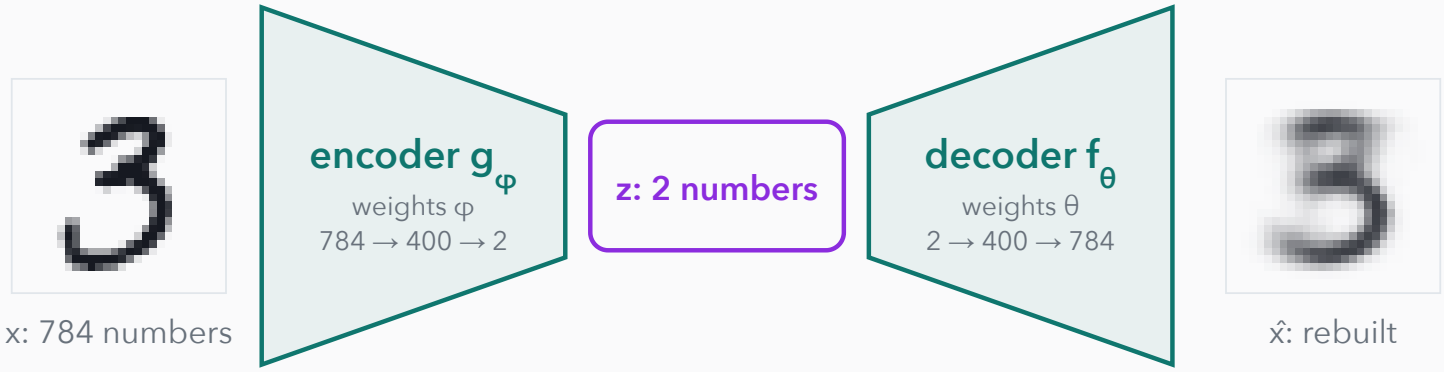
The VAE's loss for one image:

$$\underbrace{\frac{1}{2\sigma^2}\left\|x-\hat{x}\right\|^2}_{\text{rebuild it}}+\underbrace{\mathrm{KL}\left(q_\phi(z\mid x)\,\|\,\mathcal{N}(0,I)\right)}_{\text{Lecture 1's KL to the prior}}$$

> Then the VAE. Its loss is minus Lecture 1's ELBO: a rebuild term plus a KL to the prior. The hour ends there.

▶ Notebook

Part 2 · Autoencoders: squeeze, then rebuild



2.1 An image is 784 numbers ▶
2.2 Encoder, code, decoder ▶
2.3 The loss, and training ▶
2.4 What the code holds ▶

> Squeeze each digit into 2 numbers, then rebuild it from them. Two small networks, trained together on one squared-error loss.

▶ Notebook 2.1

MNIST: 70,000 handwritten digits, 28 × 28 pixels each



0	0	0	0.3	0.8	1.0	1.0	1.0	1.0	1.0	1.0
0	0.0	0.5	1.0	1.0	0.9	0.6	0.3	0.2	0.5	1.0
0	0.4	1.0	0.9	0.6	0.1	0	0	0.0	0.5	1.0
0	0.1	0.6	0.1	0	0	0	0	0.2	1.0	1.0
0	0	0	0	0	0.0	0.4	0.6	0.9	1.0	0.9
0	0	0	0	0	0.3	1.0	1.0	1.0	0.9	0.2
0	0	0	0.6	0.9	1.0	1.0	1.0	1.0	1.0	0.4
0	0	0	0.9	1.0	1.0	1.0	0.7	1.0	1.0	1.0
0	0	0	0.0	0.2	0.0	0.0	0	0.2	0.7	1.0
0	0	0	0	0	0	0	0	0	0.1	0.9

60,000 for training, 10,000 for testing.

Our running example: test digit #90, a 3. Each pixel is a number: 0 is paper, 1 is full ink.

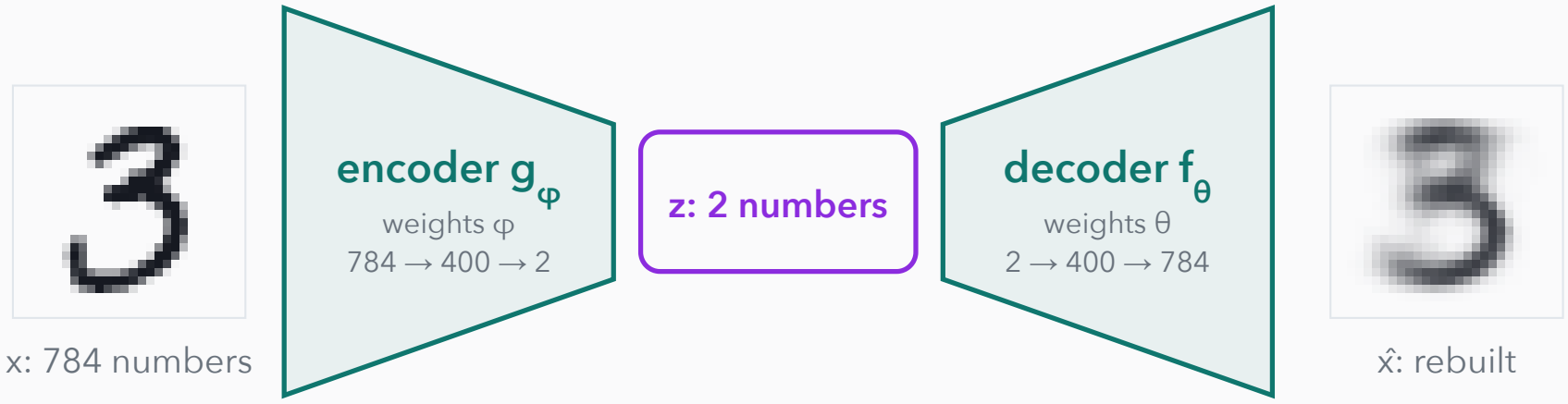
$$x \in [0, 1]^{784}$$

> We follow one digit all hour: test digit #90, a 3. To a network it is a list of 784 numbers between 0 and 1.

MNIST (LeCun, Cortes and Burges), grey levels scaled to [0, 1].

▶ Notebook 2.2

An autoencoder: encoder, short code, decoder



The decoder f_θ, with weights θ, mirrors it: 2 → 400 → 784, then a sigmoid. Train both so that $\hat{x} \approx x$.

▶ Notebook 2.2

Two small networks in PyTorch

```python
class AutoEncoder(nn.Module):
    def __init__(self, z=2):
        super().__init__()
        self.encoder = nn.Sequential(  # g_φ, weights φ
            nn.Linear(784, 400), nn.ReLU(),
            nn.Linear(400, z))
        self.decoder = nn.Sequential(  # f_θ, weights θ
            nn.Linear(z, 400), nn.ReLU(),
            nn.Linear(400, 784), nn.Sigmoid())

    def forward(self, x):
        z = self.encoder(x)     # 784 -> z numbers
        return self.decoder(z)  # z -> 784 numbers
```

Why a sigmoid at the end? Are we classifying?

No: it keeps every output pixel between 0 and 1, like the data.

> The final sigmoid squashes each of the 784 outputs into (0, 1), the range of our pixels.

▶ Notebook 2.3

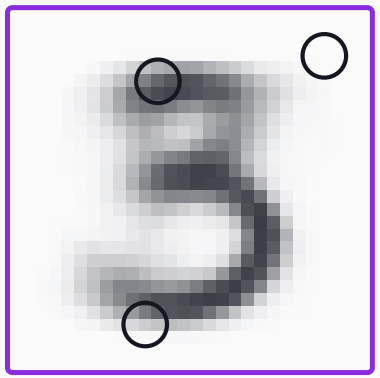
The loss: how far each rebuilt pixel is from the real one

$$L(\phi,\theta) = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{784}\left(x_{ij} - \hat{x}_{ij}\right)^2, \qquad \hat{x}_i = f_\theta\left(g_\phi(x_i)\right)$$

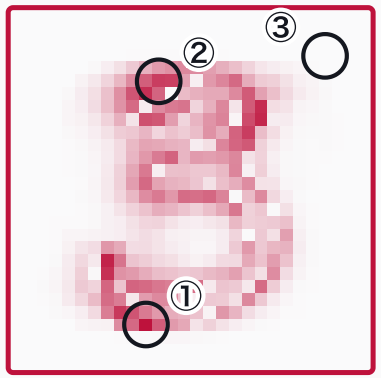
target x: our 3



prediction $\hat{x}$



$(x - \hat{x})^2$, each pixel



sum of all 784 = 26.3

pixel	x	$\hat{x}$	$(x - \hat{x})^2$
① missed ink	0.99	0.20	0.63
② extra ink	0.00	0.65	0.42
③ background	0.00	0.00	0.00
all 784 pixels			**26.3**

> Three pixels: ① missed ink costs 0.63, ② ink where the 3 has none costs 0.42, and ③ clean background costs 0.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

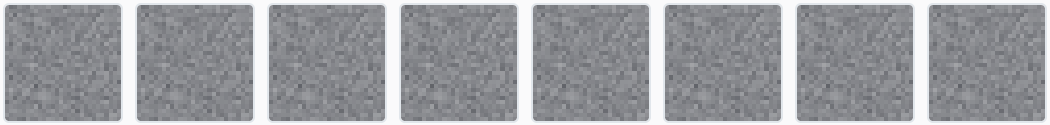
▶ Notebook 2.3

Before training: always look at what the network outputs

8 test digits



untrained output



Every output pixel is between 0.38 and 0.66: flat grey, whatever the input.

Each pixel is off by about 0.5, since the target is 0 or 1. Squared: about 0.24.

$$\underbrace{784}_{\text{pixels}} \times \underbrace{0.235}_{\text{per pixel}} \approx 185$$

> Each of 784 pixels is off by about a half, which squares to a quarter: about 185 in all. Training must bring this far down.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

One training step: forward, loss, backward, update

```
model = AutoEncoder(z=2)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(30):
    for x in loader:          # 128 images at a time
        x_hat = model(x)
        loss = ((x_hat - x)**2).sum(1).mean()
        opt.zero_grad()
        loss.backward()       # encoder AND decoder
        opt.step()
```

469 steps per pass over the 60,000 digits; 30 passes.

One loss, one backward pass: the gradient flows from the pixel errors back through the decoder, the code, and the encoder.

> loss.backward() sends gradients through the decoder, into the code, and on into the encoder: both are trained together.

▶ Notebook 2.3

Press play: reconstructions sharpen as it trains

8 test digits



what it outputs after 14,070 steps (30 epochs)

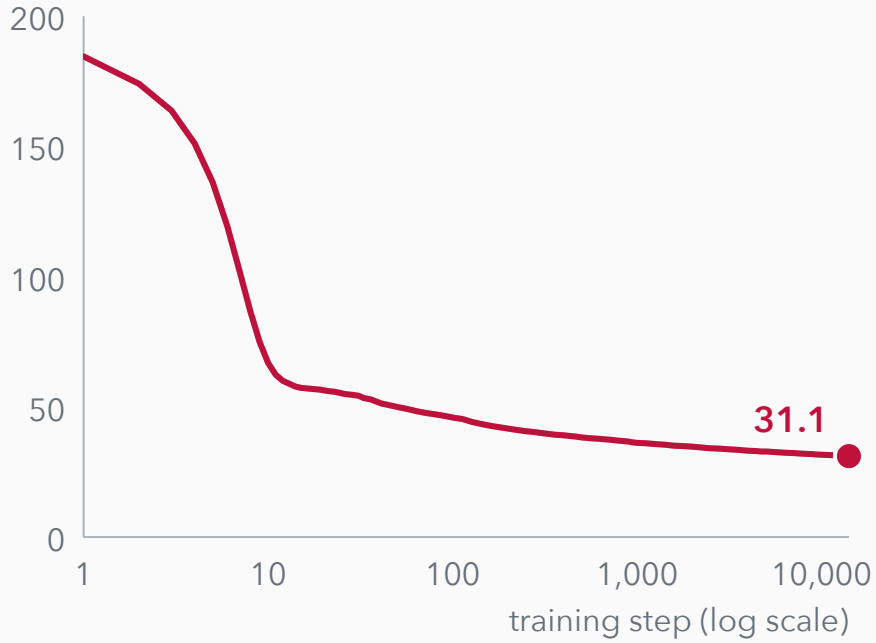


28 2 54 47 23 70 54 12

squared error of each digit (summed over its 784 pixels)

average over the 10,000 test digits: 31.1

test error while training



> The error keeps falling as each digit takes shape; after 30 epochs it is 31.1. Under each digit: its own error.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

More numbers in the code, sharper rebuilds

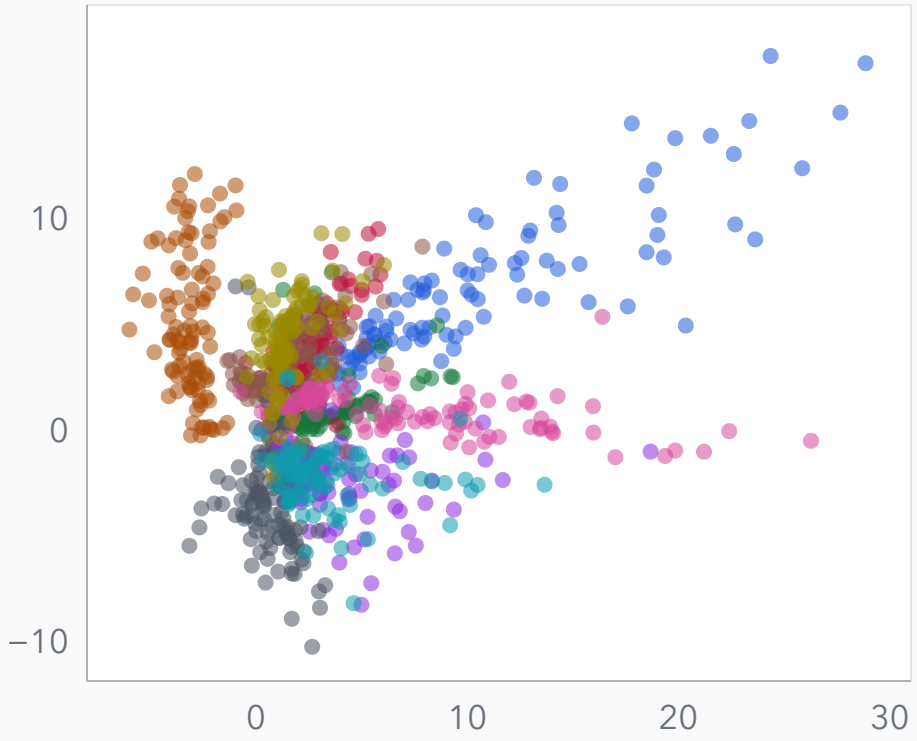


> With 32 and 128 numbers the strokes come back almost exactly. Hold on to this: Part 3 asks which of them can generate.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

Where the codes land: 1,000 test digits at their 2 numbers



0 1 2 3 4 5 6 7 8 9

the whole code: z_1 and z_2

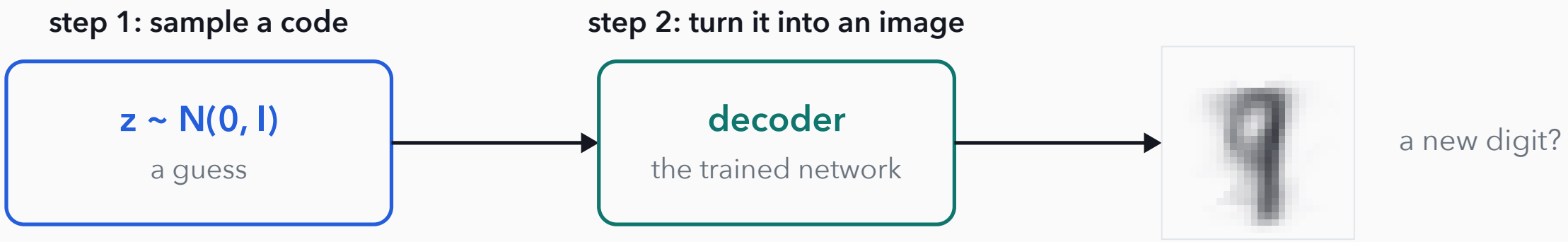
This autoencoder has exactly 2 numbers in its code, so nothing is hidden here. Hover any point to see its digit.

> Digits of one kind cluster together, although no labels were used in training. The codes run from about −5 to 22.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Part 3 · Can an autoencoder generate?



3.1 Decode random codes ▶

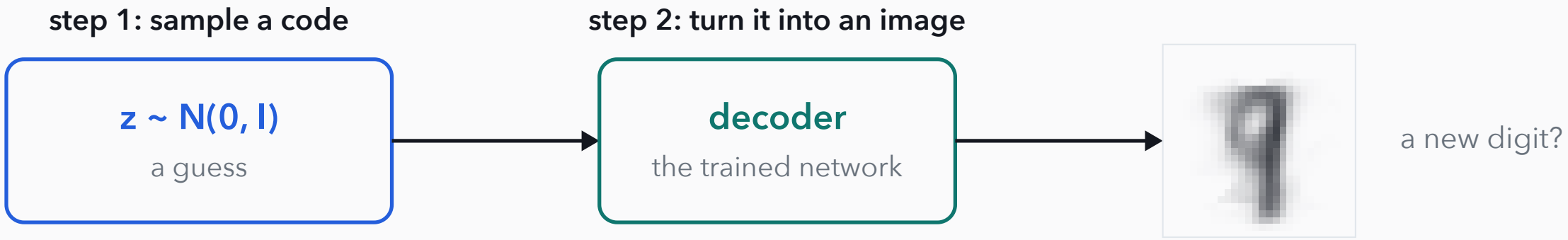
3.2 Where the codes are ▶

3.3 What we want instead ▶

> Keep the decoder, throw the encoder away, and invent the code. Does a random code give a digit?

▶ Notebook 3.1

Keep the decoder, feed it random numbers

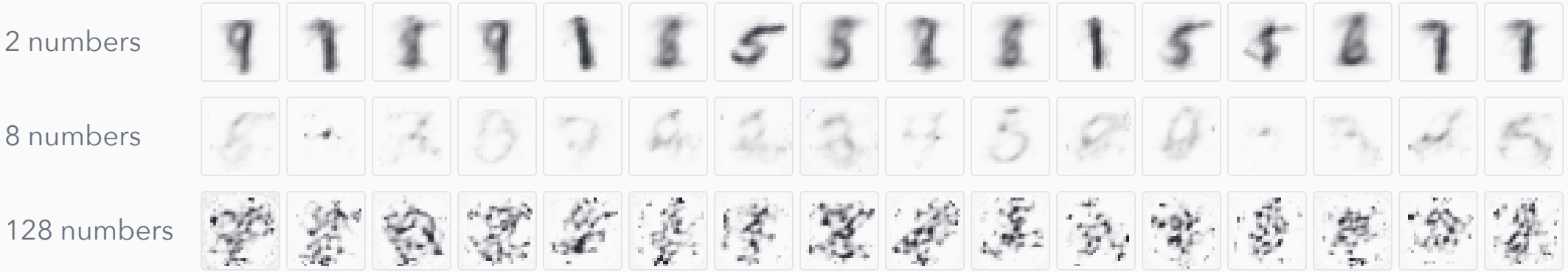


Which random numbers? The simplest: z ~ N(0, I), two independent standard bells. In code: `torch.randn(2)`.

> We need a distribution to draw codes from. Start with the simplest, the standard normal N(0, I).

▶ Notebook 3.1

Random codes through three autoencoders



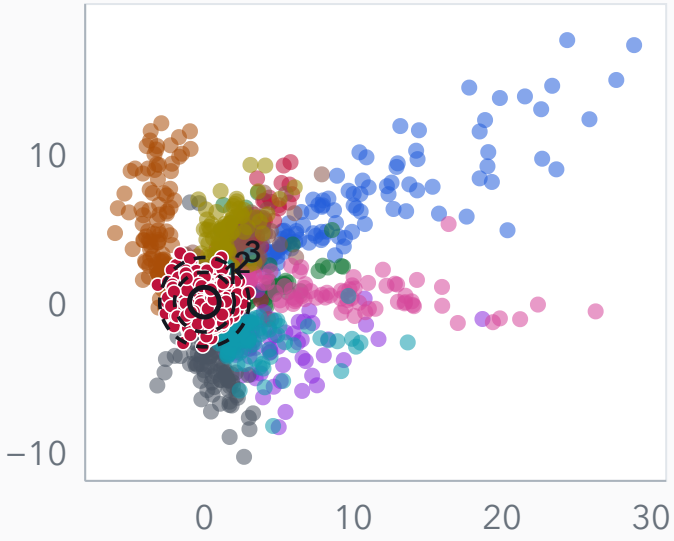
Takeaway. Better reconstruction did not give better samples: the best rebuilder makes static. Why? Look at where the codes are.

128 numbers, the best rebuilder of all, gives static.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Put the random codes on top of the real ones



0 1 2 3 4 5 6 7 8 9

real codes and 500 random ones (red)

Circles: N(0, I) at radius 1, 2 and 3.
Hover a dot to see the digit and what the decoder draws from that code.

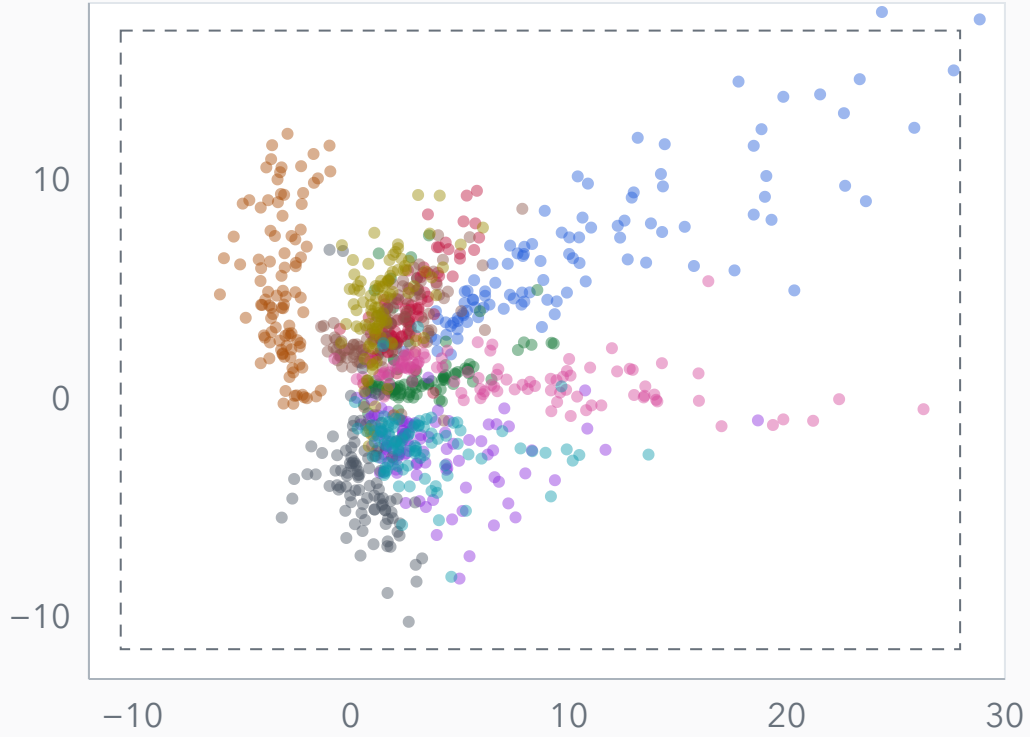
Takeaway. N(0, I) covers a small corner of the real codes: random codes reach a few digit types and miss the rest.

Nothing in the loss asked the codes to sit near N(0, I). The red draws land in one corner: some digits there, none of the rest.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Decode a whole grid of codes





Takeaway. Good digits only where real codes are. Away from them the decoder only extrapolates.

Over the real codes, digits blend into each other. Far from them, thick 1s and 0s and broken strokes: the decoder never trained there.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.3

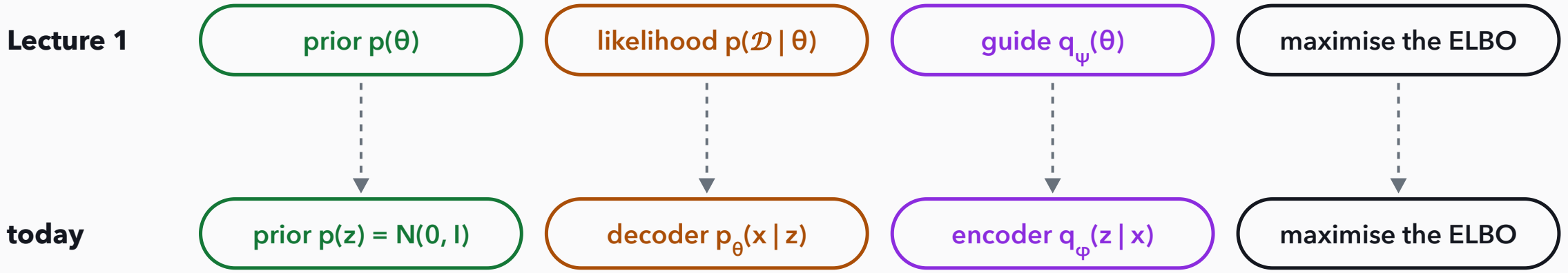
What we saw, and what we want instead

what we saw	what we want	how the VAE does it
random N(0, I) codes miss the real codes	codes that fill N(0, I), so `torch.randn` gives good codes	a KL to the prior pulls every code towards N(0, I)
gaps between codes decode to junk	each image owns a small cloud of codes, and clouds overlap	the encoder outputs a mean and a spread; z is drawn from that cloud
no way to say how likely an image is	one probability model that can score and sample	a prior and a decoder, trained with Lecture 1's ELBO

> Problem three: the autoencoder cannot score an image. We want a probability model. The VAE does all three.

▶ Notebook

Part 4 · The VAE: Lecture 1's ELBO, one image at a time



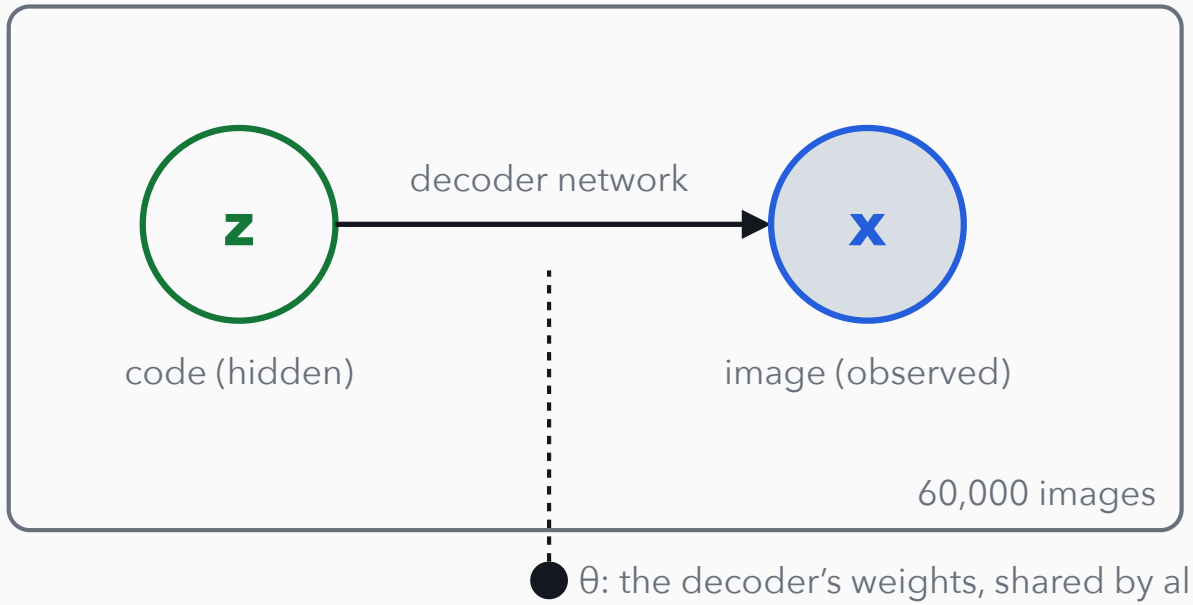
swap the names: the weights θ become a code z for each image, and the data $\mathcal{D}$ become one image x

4.1 A model with a hidden code ▶
4.2 The same wall as Lecture 1 ▶
4.3 The ELBO for one image ▶
4.4 The loss, term by term ▶
4.5 Two terms pull against each other ▶
4.6 What the VAE learned ▶
4.7 The KL weight β ▶
4.8 Wrap-up ▶

> Write a model with a hidden code, hit the same wall as Lecture 1, and get past it the same way: maximise the ELBO, one image at a time.

▶ Notebook 4.1

A generative model with a hidden code



the prior:

$p(z) = N(0, I)$, 2 numbers, fixed in advance

the likelihood:

$p(x \mid z) = N(x; \text{decoder}(z), \sigma^2 I)$

each pixel: the decoder's value + noise of size σ

> The decoder's weights θ sit outside the box: one set, shared by every image, learned from the data.

▶ Notebook 4.1

To draw an image, follow the arrows: first z, then x

① draw z

```
z = torch.randn(2)
```

z = (0.82, −0.74)

→

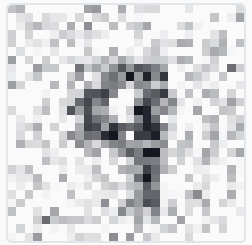
② decode



mean image $f_\theta(z)$

→

③ add pixel noise



x = mean + 0.2 × noise

Takeaway. We usually show the mean image: the pixel noise is independent speckle and adds nothing about the digit.

Step three: add independent noise of size σ = 0.2 to each pixel. That is a true sample of x; we display the mean image.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.1

The likelihood, in numbers: a squared error

$$\log p_\theta(x \mid z) = \sum_{j=1}^{784} \log \mathcal{N}\left(x_j;\ f_\theta(z)_j,\ \sigma^2\right)$$

$$= -\frac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2 \ - \ 784 \log\left(\sigma\sqrt{2\pi}\right)$$

We use σ = 0.2, about the typical pixel error of a 2-number code: √(31 / 784) ≈ 0.2. The constant −784 log(σ√2π) is +541.4.

> σ = 0.2 is a modelling choice, matched to a typical pixel error. Section 4.7 shows what changing the balance does.

▶ Notebook 4.2

Which codes made our 3? Bayes' rule, and Lecture 1's wall

$$p(z \mid x) = \frac{p_\theta(x \mid z)\ p(z)}{p_\theta(x)}$$

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$$

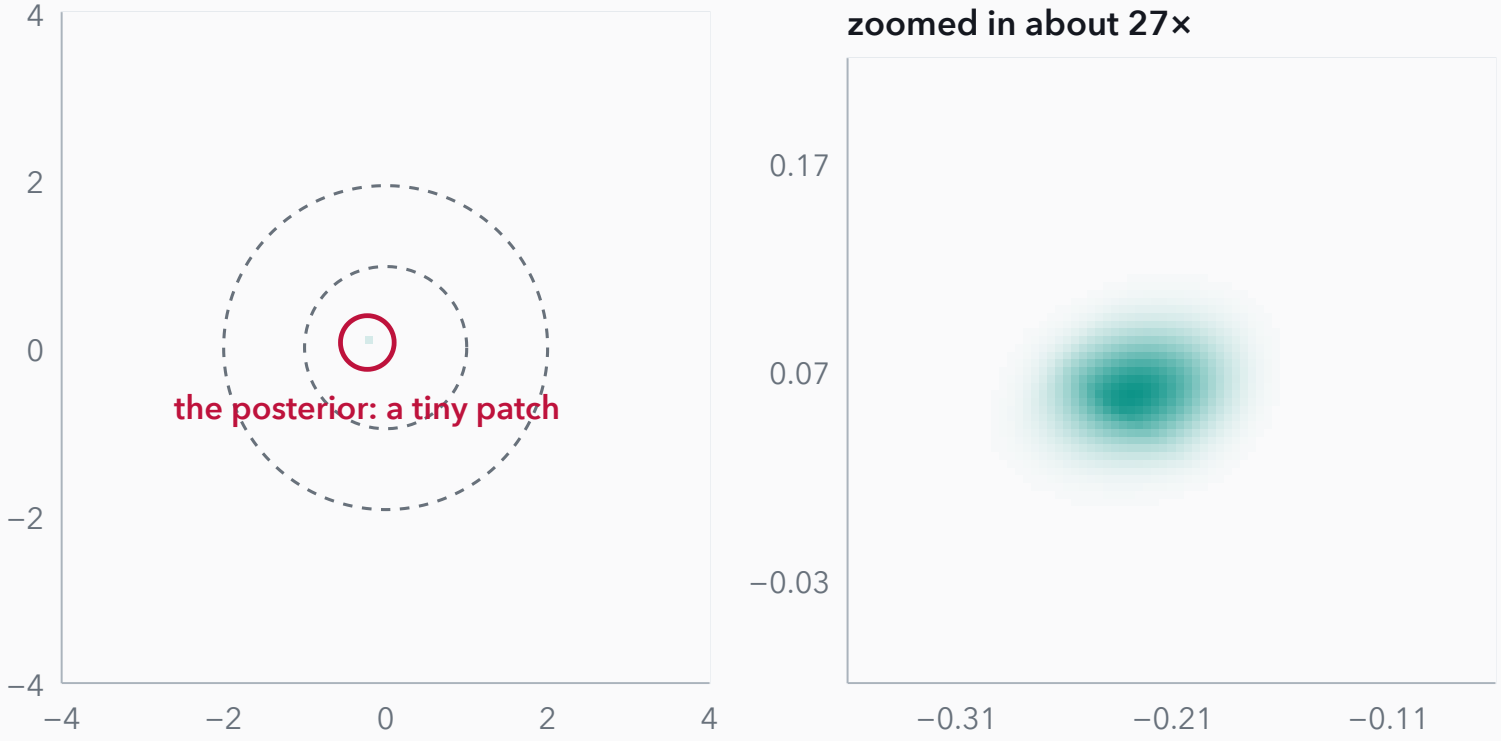
An integral over every possible code, through a neural network: no formula.

Lecture 1 hit this wall with p($\mathcal{D}$), an integral over the network's weights. We get past it the same way. ↗ Lecture 1, 1.5: approximate the posterior

> Lecture 1's problem exactly: an evidence we cannot compute. Lecture 1's answer was variational inference.

▶ Notebook 4.2

For our 3: which codes could have made it?



Centre (−0.23, 0.06), width about 0.018.

> Zoomed in: the codes that explain this 3 form a tiny patch, about 0.018 wide. With 16 numbers no grid could find it.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.2

Same problem as Lecture 1, new names

	Lecture 1	today, for one image x
the unknown, with a prior	θ: the tiny network's 7 weights	z: this image's 2-number code
prior	$p(\theta)$	$p(z) = \mathcal{N}(0, I)$
likelihood	$p(\mathcal{D} \mid \theta)$	$p_\theta(x \mid z)$: the decoder
posterior: out of reach	$p(\theta \mid \mathcal{D})$	$p(z \mid x)$
approximation	$q_\psi(\theta)$: the guide	$q_\phi(z \mid x)$: the encoder
how many q's?	one	one per image: 60,000, all computed by one encoder network

Careful with θ: today it names the decoder's weights, learned as ordinary parameters. The unknown with a prior is z.

> One change: Lecture 1 fitted one q. Here every image needs its own q, so a network, the encoder, outputs it from the image.

▶ Notebook 4.3

Lecture 1's ELBO, for one image

Lecture 1

$$\mathrm{ELBO}(q) = \mathbb{E}_{\theta\sim q}\big[\log p(\mathcal{D} \mid \theta)\big] - \mathrm{KL}\big(q(\theta) \,\|\, p(\theta)\big)$$

today: swap the names

$$\mathrm{ELBO}(x) = \mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

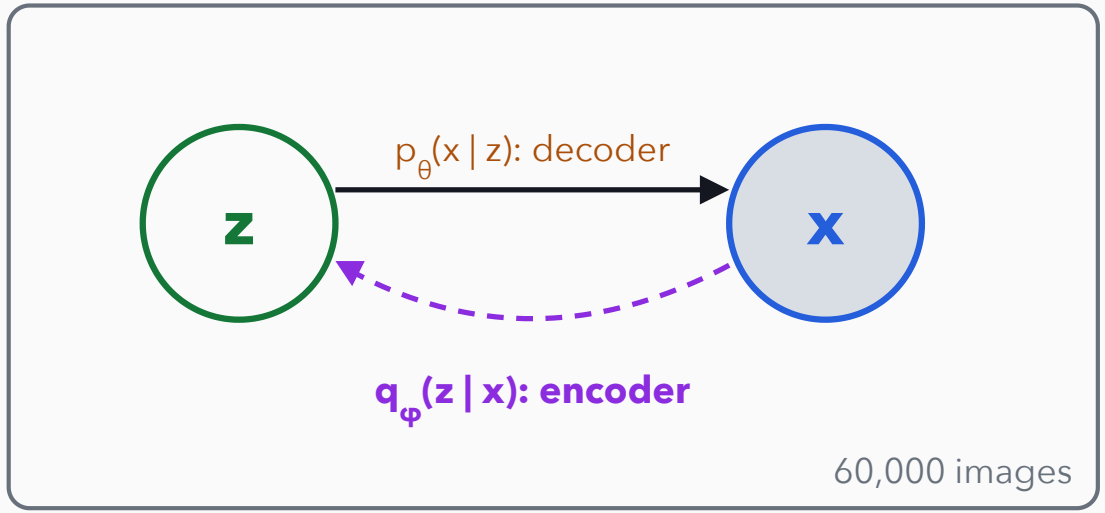
$$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big) \;\geq\; \mathrm{ELBO}(x)$$

Raising the ELBO does two jobs: θ makes the image more probable, and φ brings q closer to the posterior.

> And the same budget: log p(x) = ELBO + a KL that is never negative. Training maximises the ELBO summed over all 60,000 images.

▶ Notebook 4.3

The encoder is q: one network gives q for every image



the posterior p(z | x): which codes made this x?

it needs p(x), the integral of Section 4.2

approximate it, as in Lecture 1, with

$q_\varphi(z \mid x) = N(\mu_\varphi(x), \sigma_\varphi(x)^2)$, a Gaussian whose mean and spread the encoder computes from x

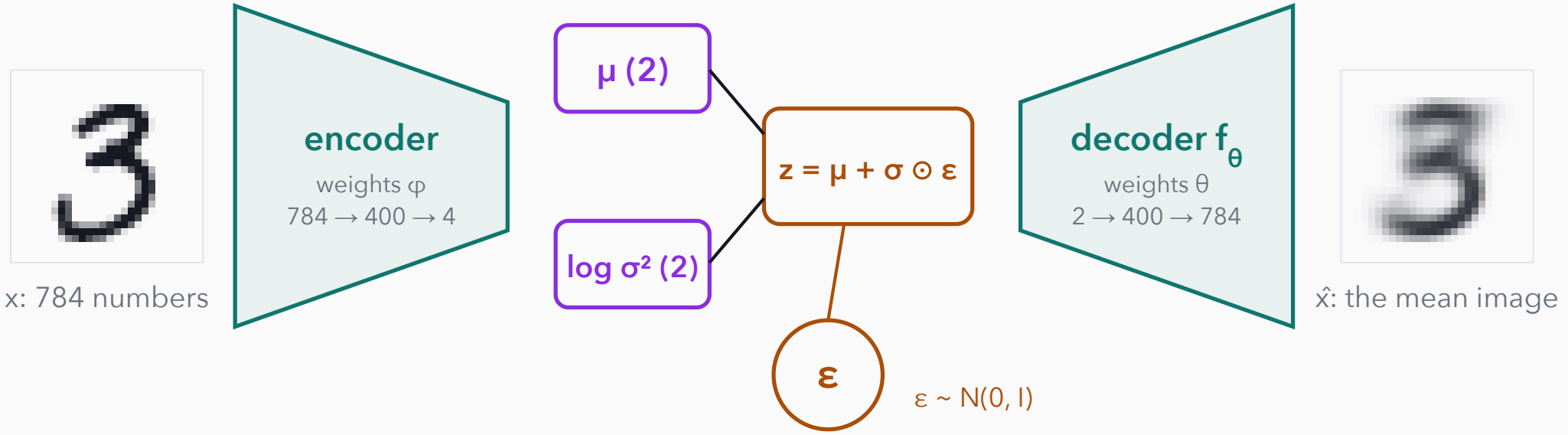
train θ and φ together: maximise the ELBO

summed over the 60,000 images

> Decoder θ and encoder φ are trained together, by one objective: the ELBO, summed over all images.

▶ Notebook 4.3

The network: an encoder that outputs a cloud, then tool ①



The decoder is the autoencoder's, unchanged: from z to a mean image.

▶ Notebook 4.4

The loss for one image is minus the ELBO

$$-\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_q\left[\tfrac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2\right]}_{\text{from } -\log p_\theta(x|z)} + \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big)}_{\text{stay near the prior}} + \underbrace{\text{const}}_{-541.4}$$

$$\mathbb{E}_q\big[\cdots\big] \approx \tfrac{1}{2\sigma^2}\left\|x - f_\theta(\mu + \sigma \odot \epsilon)\right\|^2, \qquad \epsilon \sim \mathcal{N}(0, I) \text{ (one draw, tool ①)}$$

$$\mathrm{KL} = \tfrac{1}{2}\sum_{k=1}^{2}\left(\mu_k^2 + \sigma_k^2 - \log\sigma_k^2 - 1\right) \qquad \text{(tool ②, no sampling)}$$

> The KL between two diagonal Gaussians has a formula, so it needs no sampling. Both of Lecture 1's tools, in one loss.

▶ Notebook 4.4

One step for our 3, with numbers

$$\mu = (-0.211,\ 0.070), \quad \sigma = (0.013,\ 0.010), \quad \epsilon = (0.5,\ -1.0)$$

$$z = \mu + \sigma \odot \epsilon = (-0.204,\ 0.060)$$



vs



$$\frac{1}{2(0.2)^2} \times 23.18 = 289.8$$

$$\mathrm{KL} = 7.94 \qquad L = 289.8 + 7.94 = 297.7$$

Add the KL to the prior, from tool ②: the loss for this image on this step.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.4

The VAE in PyTorch: three changes to the autoencoder

```
class VAE(AutoEncoder):                # same decoder
    def __init__(self, z=2):
        super().__init__(z)
        self.encoder[-1] = nn.Linear(400, 2 * z)  # (1)

    def forward(self, x):
        mu, logvar = self.encoder(x).chunk(2, dim=1)
        eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps       # (2)
        return self.decoder(z), mu, logvar

x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)  # (3)
loss = (recon + kl).mean()
```

(1) the encoder outputs 2z numbers, (2) the code is a reparameterised draw (tool ①), (3) the loss adds the KL (tool ②).

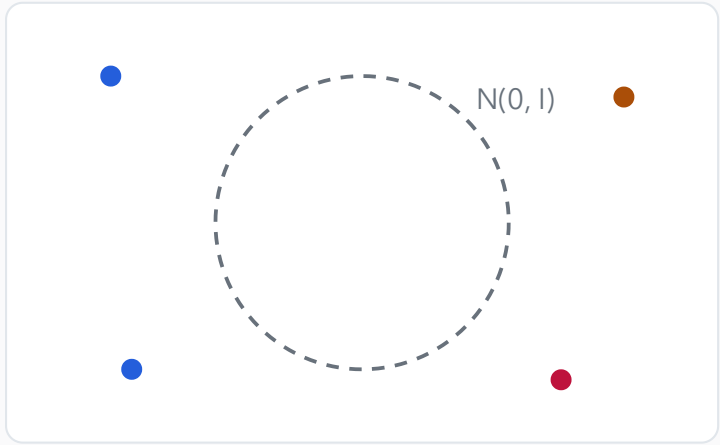
The training loop is the autoencoder's, unchanged.

> Everything else, the batches, Adam and the passes over the data, is the autoencoder's loop.

▶ Notebook 4.5

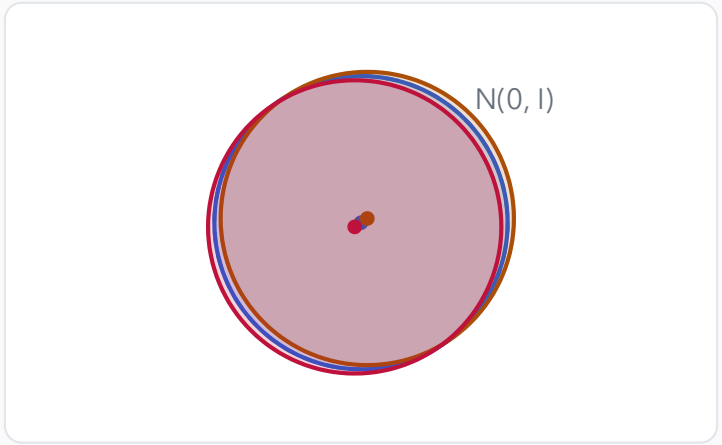
Two terms pull in opposite directions

reconstruction alone



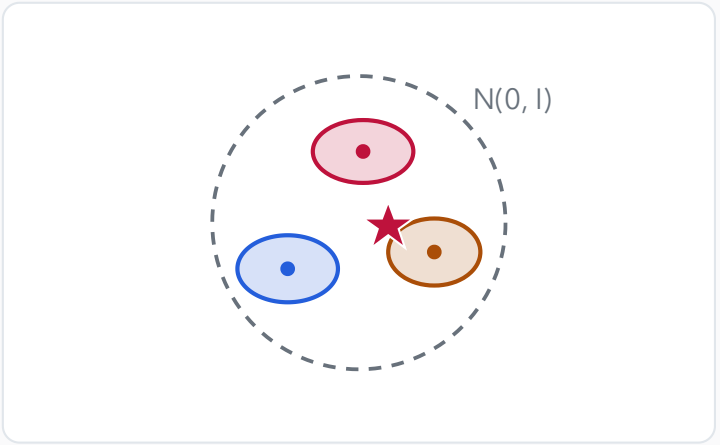
tiny clouds, far apart:
an autoencoder again

KL alone



every image gets the same cloud N(0, I):
z says nothing about the image

both together



clouds near the centre, still apart:
a random z (star) lands near some cloud

Takeaway. A VAE is an autoencoder whose codes are noisy clouds held near N(0, I). To generate, draw z ~ N(0, I) and decode.

So a random draw from N(0, I) usually lands on or near some cloud, and the decoder has been trained to draw something there.

▶ Notebook 4.6

Now sample: z ~ N(0, I), then decode

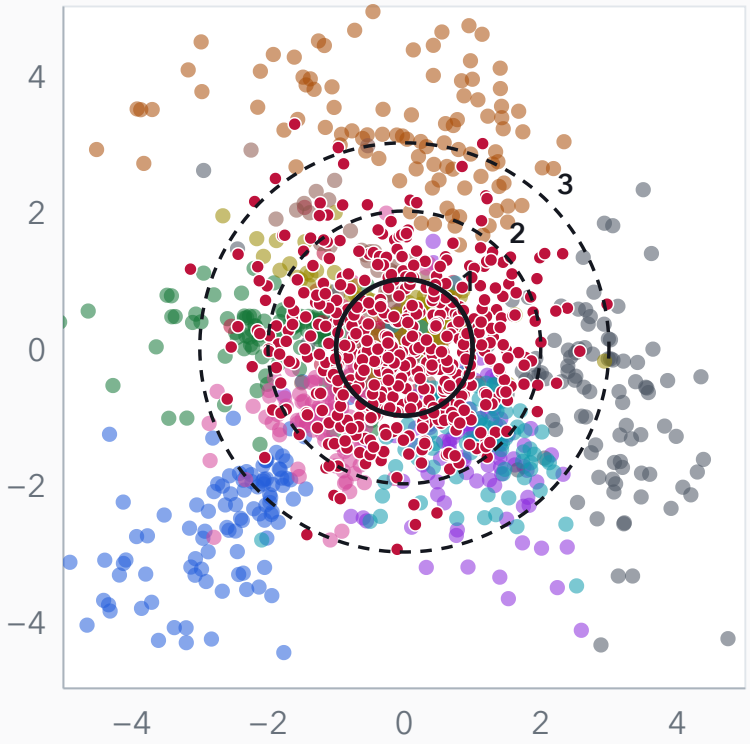


> Through the 16-number VAE: sharper and more varied. The fakes at the start of the hour came from this model.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.6

The codes now fill N(0, I)



Codes of all 10,000 test digits: mean (−0.2, −0.2), spread (1.8, 1.7).

Random draws (red) now land among real codes, on every digit.

> Overlay 500 draws from N(0, I): they fall on real codes of every digit. That is why sampling works.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.6

Decode a grid of codes: digits everywhere

VAE: codes from −3 to 3 in each direction



walk in a straight line from a 1 to a 0



the same walk, 16-number VAE



> Walk from a 1 to a 0: every step is a plausible digit. The space between images was trained, because each image owned a cloud.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.7

Put a weight on the KL: β

$$L_\beta = \mathbb{E}_q\left[\frac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2\right] + \beta\,\mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right)$$

β = 1 is the true ELBO. Changing β is the same as assuming noisier or cleaner pixels, a different σ.

Small β

codes free to spread: close to an autoencoder

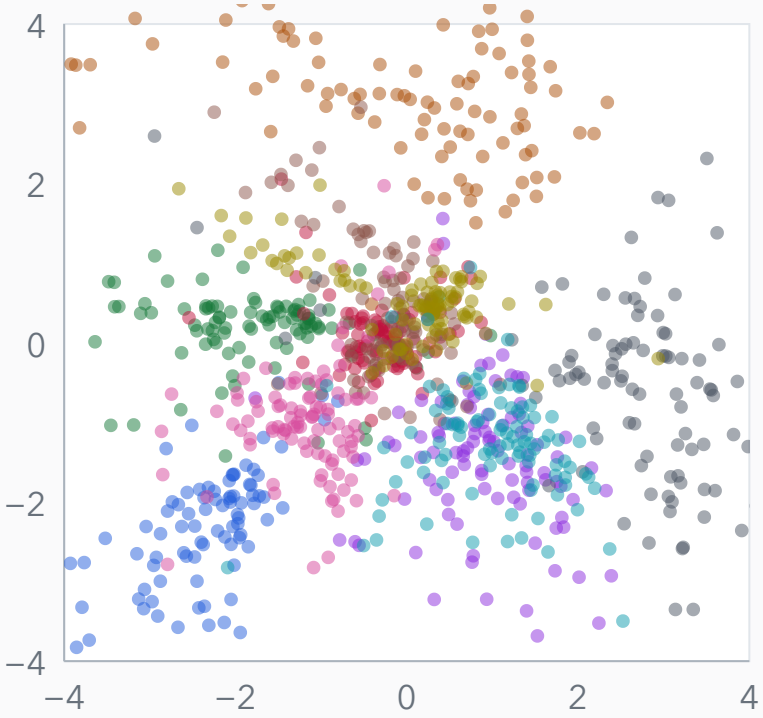
Large β

codes squeezed towards the prior: closer to N(0, I), less detail

> Predict before we look: what happens at β = 0, and at very large β?

▶ Notebook 4.7

Seven VAEs, β from 0 to 256: try them

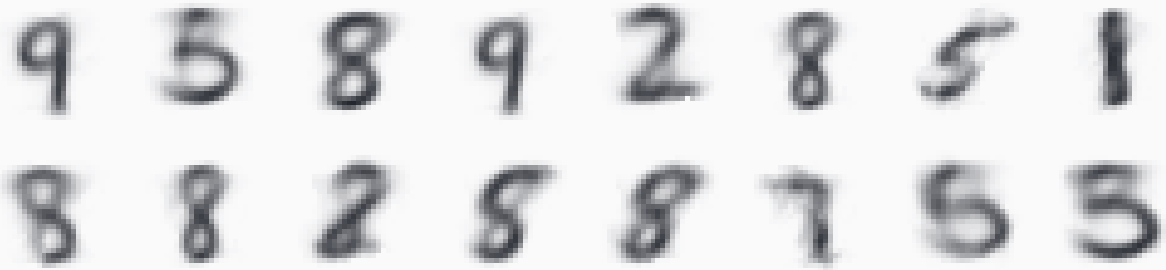


β = 1 KL 9.0 · squared error 30.8 · average σ of q 0.034

reconstructions of 8 test digits



16 samples: z ~ N(0, I), then decode



distance between two 0s ÷ distance from a 0 to a 1: 0.23

Click through β. Watch how spread the codes are, the reconstructions, and the samples.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.8

Autoencoder and VAE, side by side

	autoencoder	VAE
the encoder outputs	a code z: 2 numbers	a cloud: μ and log σ^2, that is $q_\phi(z \mid x)$
the decoder gets	z	a draw $z = \mu + \sigma \odot \varepsilon$ (tool ①)
the loss	squared error	squared error / $2\sigma^2$ + KL to N(0, I) (tool ②) = −ELBO + const
random z ~ N(0, I) gives	mostly junk	digits
can it score an image?	no	yes, through the ELBO, a lower bound on log p(x)

Takeaway. A VAE is an autoencoder trained with Lecture 1's objective. The KL to the prior makes the codes fill N(0, I), so random codes decode to digits.

Both changes come from Lecture 1: the ELBO with a code per image, the reparameterisation trick, and the Gaussian KL.

▶ Notebook 4.8

Next: can we drop the pixel likelihood altogether?

VAE samples are soft because each pixel is an independent Gaussian around a mean image.

Next: a generator trained against a second network that judges real from fake (GANs). Then many small denoising steps (diffusion).

> Lectures 3 and 4 replace it: a learned critic, then repeated denoising.

▶ Notebook

Did the autoencoder memorise? Training against test error

2-number code	training error	test error
autoencoder	30.3	31.1
PCA		43.6

Test digits were never used for training.

Training and test errors are close: it learned digits in general, not these particular 60,000.

> The gap is small (30.3 against 31.1), and the autoencoder beats PCA by a wide margin with the same 2 numbers.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Without the ReLUs and the sigmoid, an autoencoder rediscovers PCA



Test error: linear autoencoder 43.8, PCA 43.6.

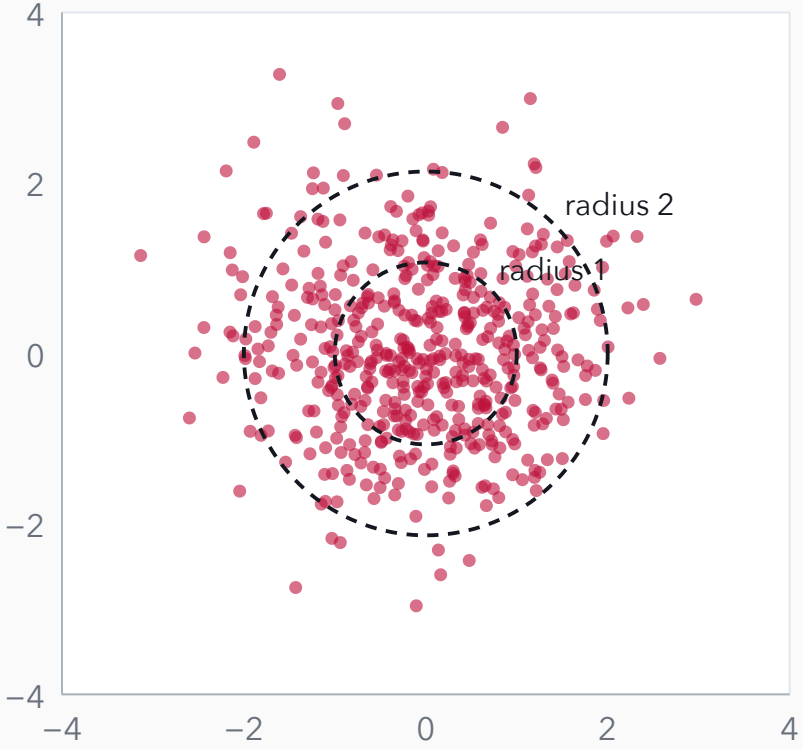
Its decoder spans almost the same plane as PCA's two directions (cosines of the angles: 1.000 and 0.92).

> Gradient descent found PCA's plane on its own: an autoencoder is PCA with bendable maps.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

N(0, I) in two dimensions: two independent standard bells



$$z_1 \sim \mathcal{N}(0,1), \quad z_2 \sim \mathcal{N}(0,1), \text{ independent}$$

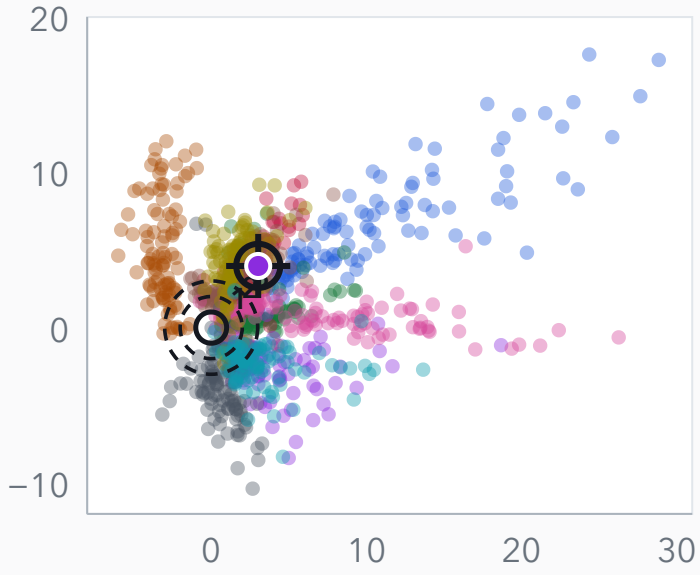
Of these 500 draws, 39% land within distance 1 of the origin and 88% within 2 (in theory 39% and 86%).

> Most draws land close to the origin. That is all torch.randn does.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

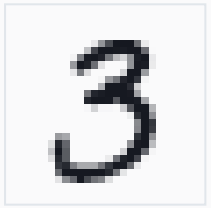
Choose a code yourself: the autoencoder



0 1 2 3 4 5 6 7 8 9

decoder of the autoencoder, run in your browser





circles: N(0, I)
at radius 1, 2, 3

the digit we encoded

z = (3.02, 3.97)
the code of our 3, decoded

> Draw a random code from N(0, I), encode a real test digit, or click anywhere on the plane. The decoder runs live in the browser.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

The autoencoder: walk in a straight line from a 1 to a 0



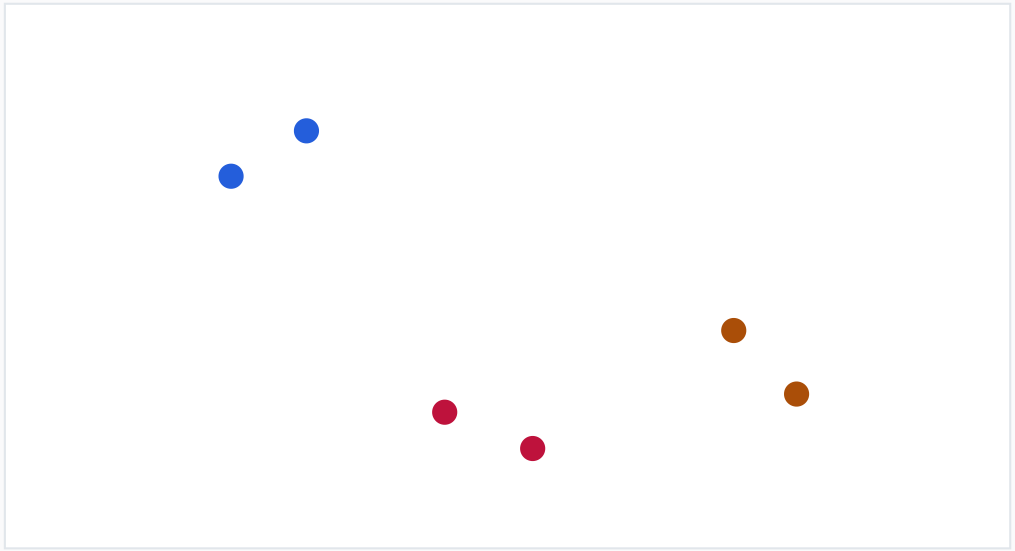
> With 2 numbers the path crosses the clusters of other digits; with 16 the middle passes through blobs that are no digit.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

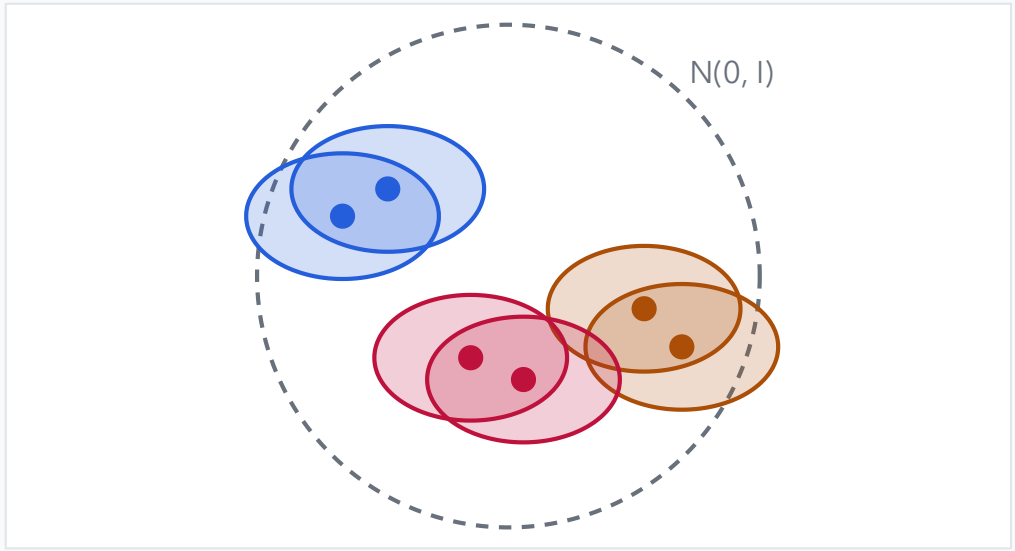
Another road to the same loss: points become clouds

autoencoder: each image → one point



points: the space between them is never trained

VAE: each image → a small cloud



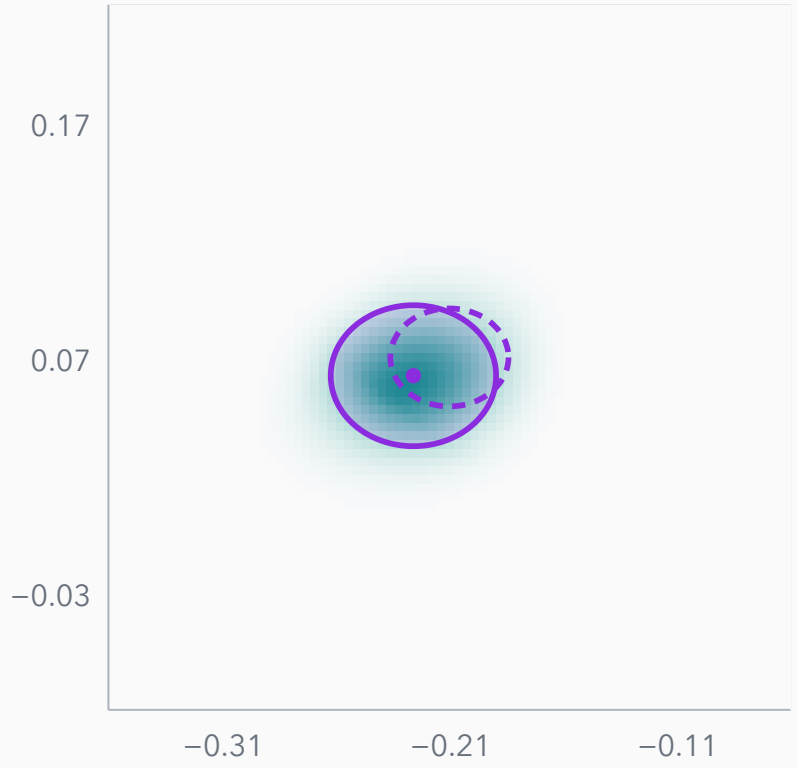
clouds overlap: the space between images is trained

A sketch, not data. Colours: three kinds of digit.

> Send each image to a small cloud and decode a random point from it, with a KL that keeps clouds near N(0, I). That is the VAE's loss again.

▶ Notebook

Check the encoder: its q for our 3, against the true posterior



	value
log p(x), the exact target	244.07
ELBO, q fitted to this image alone	244.05
ELBO, q from the encoder (one pass)	243.37

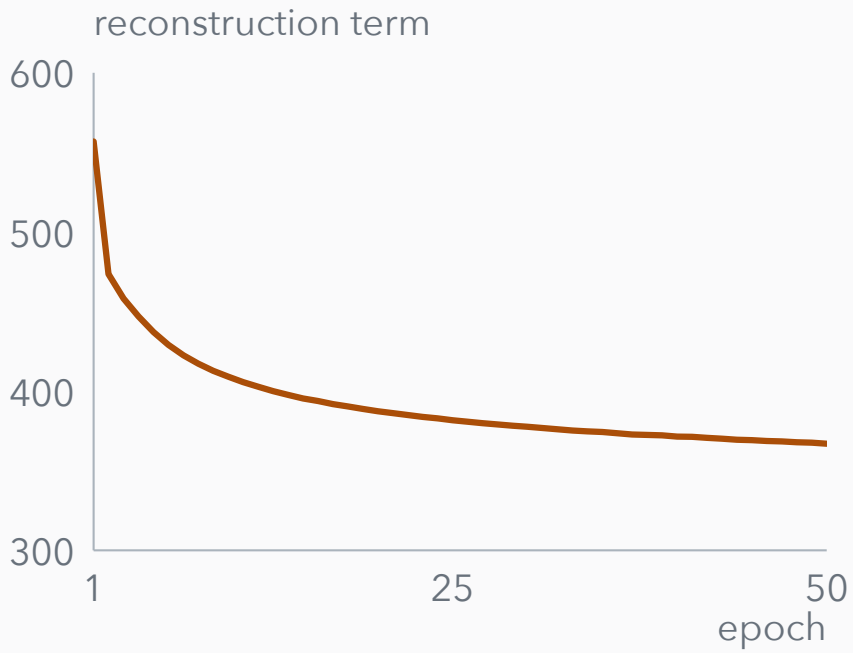
Dashed: the encoder's q. Shaded: the per-image fit. Both 2 standard deviations.

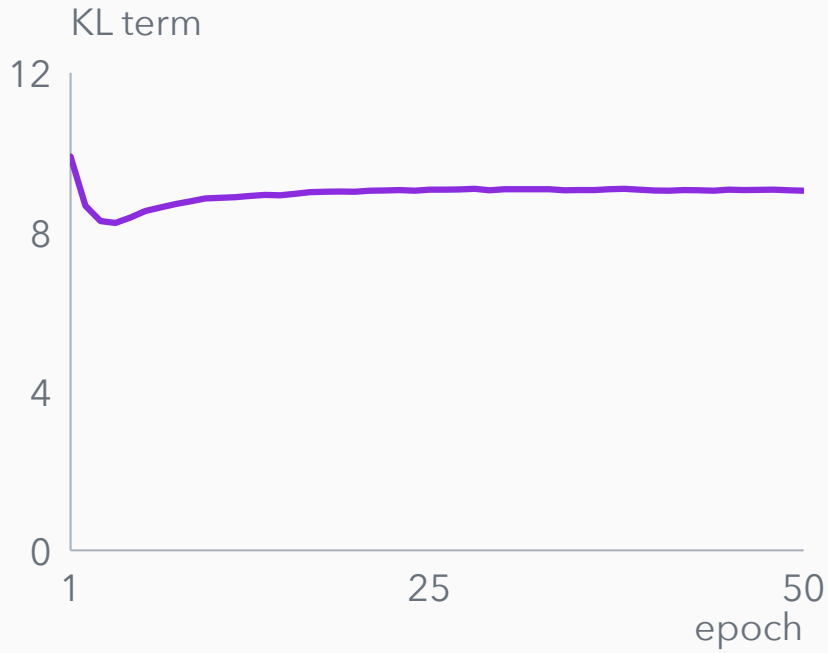
> The encoder's q sits slightly off-centre and a little too narrow: the price of one network serving all 60,000 images.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

While training: reconstruction falls, the KL settles



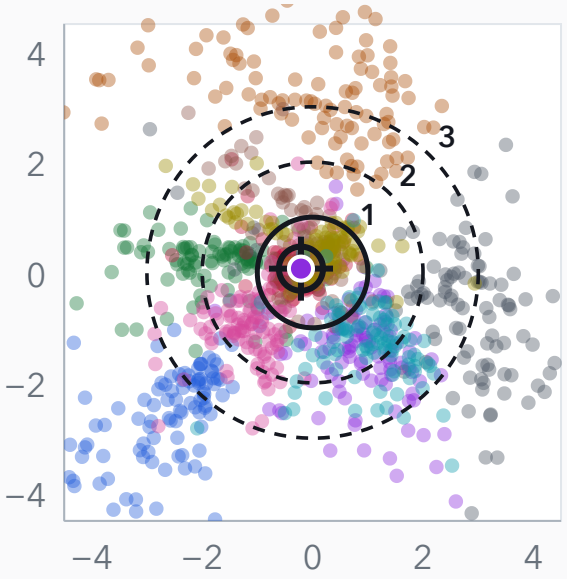


> After 50 epochs: reconstruction 367, KL 9.0 per image. The KL never falls to zero: then the code would say nothing about the image.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

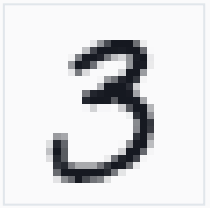
Choose a code yourself: the VAE against the autoencoder



0 1 2 3 4 5 6 7 8 9

decoder of the VAE, run in your browser





the digit we encoded

circles: N(0, I)
at radius 1, 2, 3

z = (−0.21, 0.07)
the code of our 3, decoded

> Random codes, encoded test digits, or click anywhere. Switch between the VAE and the autoencoder to compare.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 0: is it just an autoencoder?

β	KL per image	average σ of q	spread of codes
0	68.3	0.0001	5.36
1	9.0	0.0343	1.68
256	0.0	1.0001	0.00

With β = 0 nothing stops q from shrinking.

It still draws $z = \mu + \sigma\varepsilon$, unlike an autoencoder. But training drives σ to 0.0001, so it behaves like one.

> But training drives σ almost to zero, and then it behaves like one: spread-out codes and poor samples.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 256: every code decodes to the same blur

β = 256, rebuilt



β = 256, samples



KL = 0.00: q(z | x) equals the prior for every image.

The decoder learns to ignore z and outputs the average digit. This is called **posterior collapse**.

> Then z carries no information, so the decoder ignores it and draws the average digit every time.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).